CIS*3110 Operating Systems

Week 12

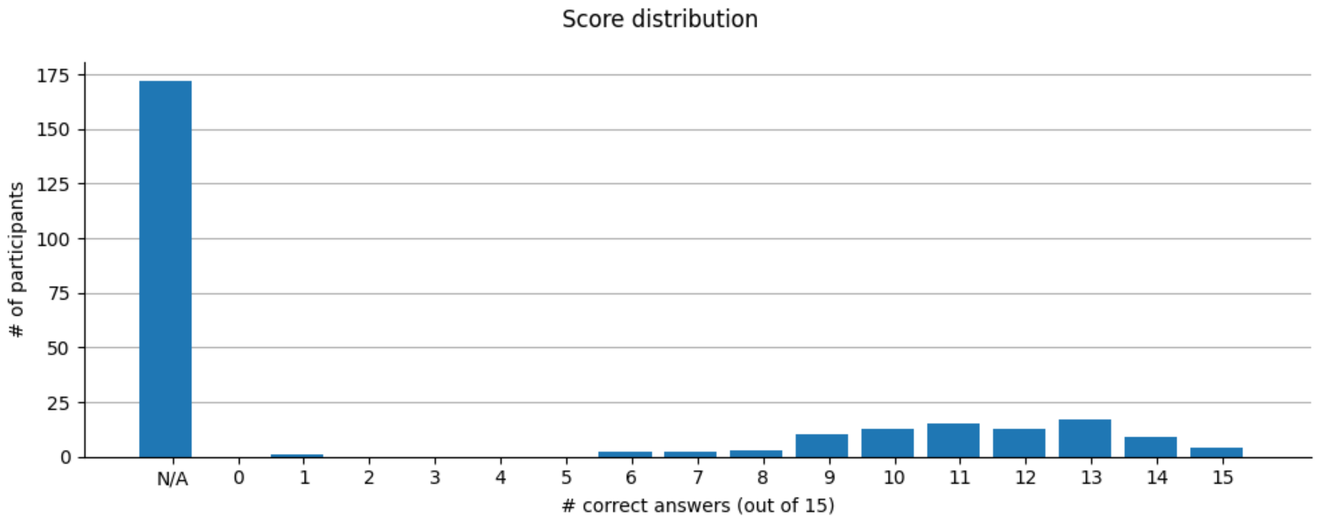
Lecture #19 Files and directories

Prof. Yulia Kotseruba

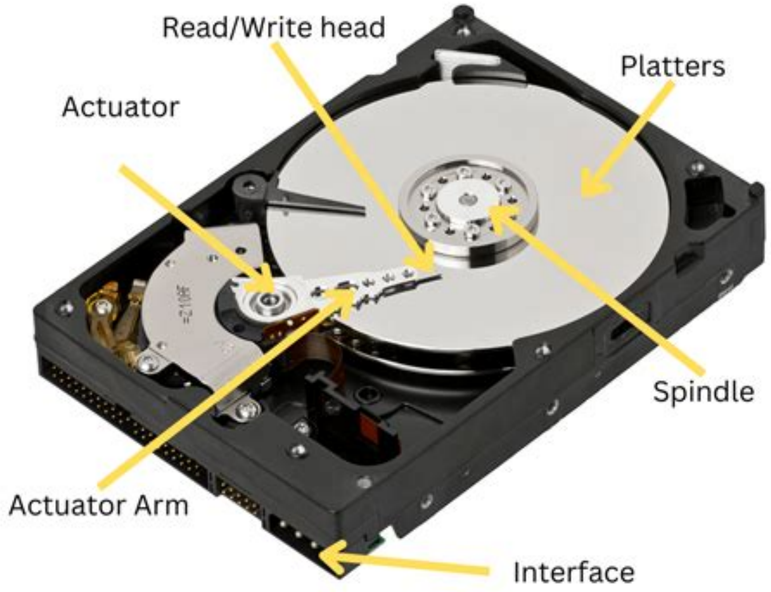
March 24, 2026

Quiz 17

- 68% average (up from 64% in quiz 16)
- 3 hardest questions:
- (50%) What does the rightmost node in the CFS's red-black tree represent? A process that … a) will run next b) just finished running c) has the highest vruntime d) has the lowest nice value
- (55%) What is true about multi-level paging? Select all that apply a) eliminates internal fragmentation b) eliminates external fragmentation c) incurs memory overhead d) less memory overhead
- (60%) In Peterson's algorithm, flag[0]=1 and flag[1]=0 indicates that a) thread 0 entered CS b) thread 1 entered CS c) thread 0 will enter CS d) thread 1 will enter CS



Recap



- Hard drive is the most common I/O device
  - consists of **rotating platters** and an **arm** with head
  - reading/writing from HD incurs **latency**
    - **seek** - moving the arm to the right track
    - **rotational** - rotating the platter to put the correct sector under the head
    - **transfer** - reading/writing data
- HD benchmarking
  - **random** workload
    - write small amount of data to random locations - tests **avg positioning time**
  - **sequential** workload
    - read a continuous chunk of data from any location - tests **transfer time**

Recap

- I/O scheduling - prioritize **short I/O jobs first** (approximate SJF scheduling)
  - SSTF - shortest **seek time** first
    - sort requests by the seek time (track) -> can starve
  - SCAN (aka elevator)
    - move arm (sweep) **back and forth across tracks** and serve requests
    - F-SCAN - freeze requests during sweep to prevent starvation
    - C-SCAN - serve requests only in one direction to avoid favouring middle tracks
  - SPTF - shortest **positioning time** first
    - take into account seek + rotation

Recap

- RAIDs - Redundant Arrays of Inexpensive Disks
  - RAID-0 - no redundancy, upper bound on performance
    - **striping** - distribute data across drives for better parallelism
  - RAID-1 - better redundancy, worse performance than RAID-0
    - **mirroring** - duplicating data for redundancy
      - N/2 drives are available for storage
  - RAID-4/5 - redundancy same(-ish) as RAID-1 but better performance
    - **parity** - compute XOR across blocks and store on a separate drive
      - N-1 drives available for storage

RAID-0 striping

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

RAID-1 mirroring

Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

RAID-4 parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

RAID-5 rotational parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

Files and directories

- File is a **sequence of bytes** with
  - name - what the users see, e.g. `lecture19.pdf`
  - inode - data structure that the OS sees
    - has a different id, e.g. `55391937`
- Directory is also a file with
  - name
  - inode
    - contents is the list of other files and directories in it

Directory tree

- Directories can contain files and other directories
- Directories form a **tree**
  - **root directory** is at the top of the tree
  - called **'/'** in Linux
    - all directories are located under '/'

this is root directory and first level of directories under it on my machine

```
yulia$ tree / -L 1
/
├── Applications
├── bin
├── cores
├── dev
├── etc
├── Library
├── opt
├── private
├── sbin
├── System
├── tmp
├── Users
├── usr
├── var
└── Volumes
```

`tree` (on Linux/Mac) will output a nicely formatted view of the directory

Name

- A **human-readable** identifier
  - **name** - an arbitrary string of chars
  - **extension** - an (optional) arbitrary string of chars
    - extensions are a **convention**, not enforced
      - common extensions: txt, c, xlsx, png, jpeg, mp3, mp4
      - extension **does not guarantee** the file is of that type
      - files can have no extension or an arbitrary extension

```
folder
├── file1.txt
├── file2.bla
├── subfolder1
│   └── file3.txt
├── subfolder2
└── subfolder3
    └── file5
```

Path

- Path tells us how to find a file
- Absolute path - works from any location
  - contains all directories from the root to the file
  - e.g. /home/yulia/folder/subfolder/file3.txt
    always starts with / (on Linux/Mac)

```
folder
├── file1.txt
├── file2.bla
├── subfolder1
│   └── file3.txt
├── subfolder2
└── subfolder3
    └── file5.txt
```

- Relative path - works from a single specific location
  - contains all directories from a specific location to the file
  - e.g. folder/subfolder1/file3.txt
    does not start with / (on Linux/Mac)

File system API

- `open` - open/create a file
- `read/write` - copy bytes from/to memory
- `lseek` - get/set current location in the file
- `close` - stop using a file
- `unlink` - delete a file
- `link, symlink` - link to a file
- `rename` - change file name
- `mkdir/rmdir` - create/delete directories

open

- `int open(const char *path, int flags, mode_t mode)`
- `path`
  - opens file - if path exists
  - creates file - if path does not exist and `O_CREAT` flag is provided
- `flags`
  - must include one of `O_RDONLY`, `O_WRONLY`, or `O_RDWR`
  - optional flags
    - `O_CREAT` - create file
    - `O_TRUNC` - truncates file to length 0 (empties it)
- `mode` - permissions for user, group, other
- on success - returns a file descriptor (an int)
- on failure - returns -1

read

- `ssize_t read(int fd, void *buf, size_t count)`
- `fd` – valid file descriptor returned by `open()`
- `buf` – buffer for bytes read from the file
- `count` - number of bytes to read
- on success
  - returns number of bytes read (>=0)
    - advances the file offset
    - next read will continue from the previous offset location
- on failure
  - returns -1, sets `errno`, unspecified if file position changes

write

- `ssize_t write(int fd, void *buf, size_t count)`
- `fd` – valid file descriptor returned by `open()`
- `buf` – buffer for bytes to write to the file
- `count` - max num of bytes to write
- on success
  - returns number of bytes written (>=0)
  - advances the file offset
  - next write will continue from the previous offset location
- on failure
  - returns -1, sets `errno`, unspecified if file position changes

lseek

- `off_t lseek(int fd, off_t offset, int whence)`
- `fd` – valid file descriptor returned by open()
- `offset` – where to move the offset
- `whence` - directive
  - `SEEK_SET` - set the offset to offset bytes
  - `SEEK_CUR` - set to its current location + offset bytes
  - `SEEK_END` - set to the file size + offset bytes
- on success
  - returns the new `offset` location (num. bytes from the beginning of the file)
- on failure
  - returns -1, sets `errno`

close

- `int close(int fd)`
- `fd` – valid file descriptor returned by `open()`
- on success
  - closes the file descriptor
- on failure
  - returns -1, sets `errno`

example #1

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("file.txt", O_RDONLY);
    char buf[4];
    ssize_t n;
    while ((n = read(fd, buf, sizeof(buf))) > 0) {
        printf("buf=%s n=%d offset=%lld\n",
          buf, n, (long long)lseek(fd, 0, SEEK_CUR));
    }
    close(fd);
    return 0;
}
```

open file to read

allocate a buffer on the stack

read 4 bytes at a time

print buffer, num bytes read, and offset

close the file

read_seq.c

file.txt

```
AAAA
BBBB
CCCC
DDDD
```

```
>>> ./read_seq
buf=AAAA n=4 offset=4
buf=
BBB n=4 offset=8
buf=B
CC n=4 offset=12
buf=CC
D n=4 offset=16
buf=DDDD n=3 offset=19
```

what will this print?

something is off,
how to change this to print every line?

example #1

```
// includes here

int main() {
    int fd = open("file.txt", O_RDONLY);
    char buf[5];
    ssize_t n;
    while ((n = read(fd, buf, sizeof(buf))) > 0) {
        printf("buf=%s n=%d offset=%lld\n",
          buf, n, (long long)lseek(fd, 0, SEEK_CUR));
    }
    close(fd);
    return 0;
}
```

open file to read

allocate a buffer on the stack

read 4 bytes at a time

print buffer, num bytes read, and offset

close the file

read_seq.c

file.txt

```
AAAA
BBBB
CCCC
DDDD
```

```
>>> ./read_seq
buf=AAAA
 n=5 offset=5
buf=BBBB
 n=5 offset=10
buf=CCCC
 n=5 offset=15
buf=DDDD
 n=4 offset=19
```

what will this print?

something is off,
how to change this to print every line?

forgot the new lines, need to read 5 chars instead

example #2

```
// includes here

int main() {
    int fd = open("file.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    const char *parts[] = {"AAAA", "BBBB", "CCCC", "DDDD"};
    for (int i = 0; i < 4; ++i) {
        ssize_t n = write(fd, parts[i], 4);
        off_t offset = lseek(fd, 0, SEEK_CUR);
        printf("n=%zd off=%lld\n", n, (long long) offset);
    }
    close(fd);
    return 0;
}
```

open the file for writing

strings to write

write each string

get current offset

print n and offset after each write

write_seq.c

```
>>> ./write_seq
n=4 off=4
n=4 off=8
n=4 off=12
n=4 off=16
```

what will this print?

what will the file look like?

file.txt

AAAABBBBCCCCDDDD

what if I want to have new lines after each string?
need to add newine char to parts and increase n to 5

example #3

```
int main(void) {
    int fd = open("file.txt", O_RDONLY);

    char buf[100];
    lseek(fd, 10, SEEK_SET);
    ssize_t n = read(fd, buf, 4);
    printf("buf=%s n=%zd offset=%lld\n",
        buf, n, (long long)lseek(fd, 0, SEEK_CUR));

    lseek(fd, 0, SEEK_SET);
    n = read(fd, buf, 100);
    printf("buf=%s n=%zd offset=%lld\n",
        buf, n, (long long)lseek(fd, 0, SEEK_CUR));

    close(fd);
    return 0;
}
```

open the file for reading

move the offset to position 10 (C in the file)

read 4 bytes into the buffer

print the buf, number of bytes read, and current offset

move the offset to the beginning of the file

read 100 bytes into a buffer

print the buf, number of bytes read, and current offset

read_nonseq.c

file.txt

AAAA
BBBB
CCCC
DDDD

```
>>> ./read_nonseq
buf=CCCC??? n=4
offset=14
buf=AAAA
BBBB
CCCC
DDDD n=19 offset=19
```

what will this print?

something is off, what are the ?? after CCCC?

read will read as many bytes are available up to the count requested

example #4

```
//includes here

int main(void) {
    int fd1 = open("file.txt", O_RDWR);
    int fd2 = open("file.txt", O_RDWR);
    char buf[5];

    n = read(fd1, buf, 5);

    n = read(fd2, buf, 5);

    n = write(fd1, "EEEE", 4);

    close(fd1); close(fd2);
    return 0;
}
```

open file for writing and reading

open file for writing and reading again

read 5 characters from file descriptor #1

read 5 characters from file descriptor #2

write 5 characters to file descriptor #1

close both and exit

write_read_copy.c

file.txt

AAAA
BBBB
CCCC
DDDD

```
>>> ./write_read_copy
```

what will the file.txt look like after this is done?

which line will EEEE be written on?

~~option #1: fd1 and fd2 share the offsets~~
~~two reads from fd1 and fd2 will push the offset to 10~~

AAAA
BBBB
EEEE
DDDD

option #2: fd1 and fd2 **do not share the offsets**
only the read from fd1 will update the offset

AAAA
EEEE
CCCC
DDDD

example #5

```
//includes here

int main(void) {
    int fd1 = open("file.txt", O_RDWR);   open file for writing and reading
    int fd2 = dup(fd1);   copy the file descriptor
    char buf[5];

    n = read(fd1, buf, 5);   read 5 characters from file descriptor #1

    n = read(fd2, buf, 5);   read 5 characters from file descriptor #2

    n = write(fd1, "EEEE", 4);   write 5 characters to file descriptor #1

    close(fd1); close(fd2);   close both and exit
    return 0;
}
```

write_read_dup.c

file.txt

AAAA
BBBB
CCCC
DDDD

now both file descriptors are the same, so file will look like this

AAAA
BBBB
EEEE
DDDD

```
>>> ./write_read_dup
```

Summary so far

- Files have
  - name - label and path
  - data - contents of the file
  - offset - current position in the file
- File descriptor - pointer to the open file
  - can read/write to files sequentially or non-sequentially by controlling the offset
  - can have multiple file descriptors for a single file
    - with independent offsets
    - with the same offset

File system implementation

- **file descriptor** - a pointer to a file
- **inode** - a struct with file information
- **block** - where inode and file data are stored on the hard drive

File descriptors

- A pointer to a file
  - an integer
  - stored in a per-process table
- File can be of **any** type
  - regular files, directories, sockets, pipes, etc…
- I/O devices are also treated as files by the OS
  - can use the same `open`/`read`/`write` system calls

inode

- aka index node (alternative spelling i-node)
- A data structure that stores **metadata** of the files/directories
  - **inode number** - a unique identifier
  - **file type** - reg file, directory, symbolic link, etc.
  - **permissions** - read, write, execute for user, group, others
  - **owner** - user ID and group ID
  - **size** of the file
  - **timestamps** - time of creation, modification, change, last access
  - **link count** - number of hard links to inode
  - **pointers** - data blocks on the storage device (next lecture)

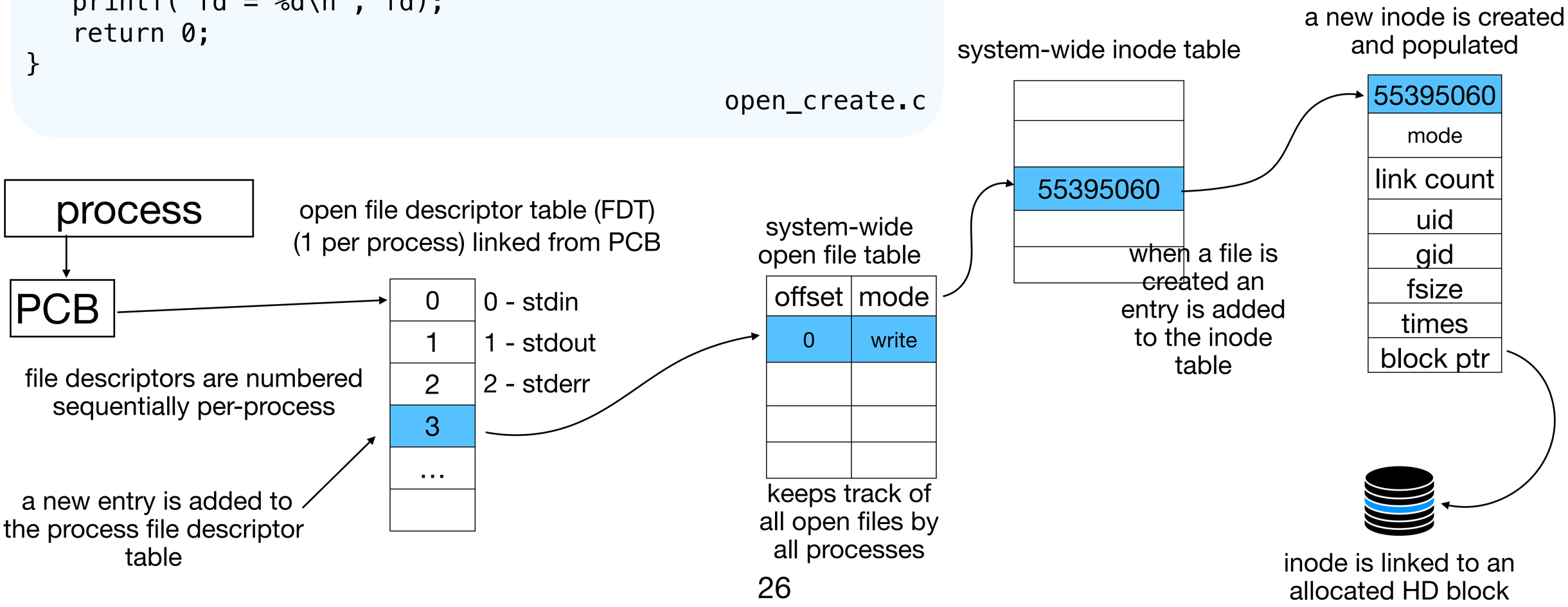
opening a file

```
int main(void)
{
   int fd = open("file.txt", O_CREAT | O_WRONLY, 0644);

   printf("fd = %d\n", fd);
   return 0;
}
```

open_create.c

```
>>> ./open_create
fd = 3
>>> ls
file.txt
```



Summary so far

- Every process has a PCB
- Every process has a file descriptor table
  - linked from the PCB
  - contains all open file descriptors (ints)
  - maps local file descriptors to the system-wide open file table
- System-wide tables
  - open file table - all files currently open, their mode and offsets
  - inode table - all inodes in the system

2 descriptors for 1 file

```
int main(void) {
    int fd1 = open("file.txt", O_CREAT | O_WRONLY, 0644);
    int fd2 = open("file.txt", O_WRONLY);
    printf("fd1 = %d\n", fd1);
    printf("fd2 = %d\n", fd2);
    write(fd1, "AAAA", 4);
    write(fd2, "BBBB", 4);
    close(fd1);
    close(fd2);
    return 0;
}
```

open two fds for the same file

print fd numbers

write AAAA into the first fd

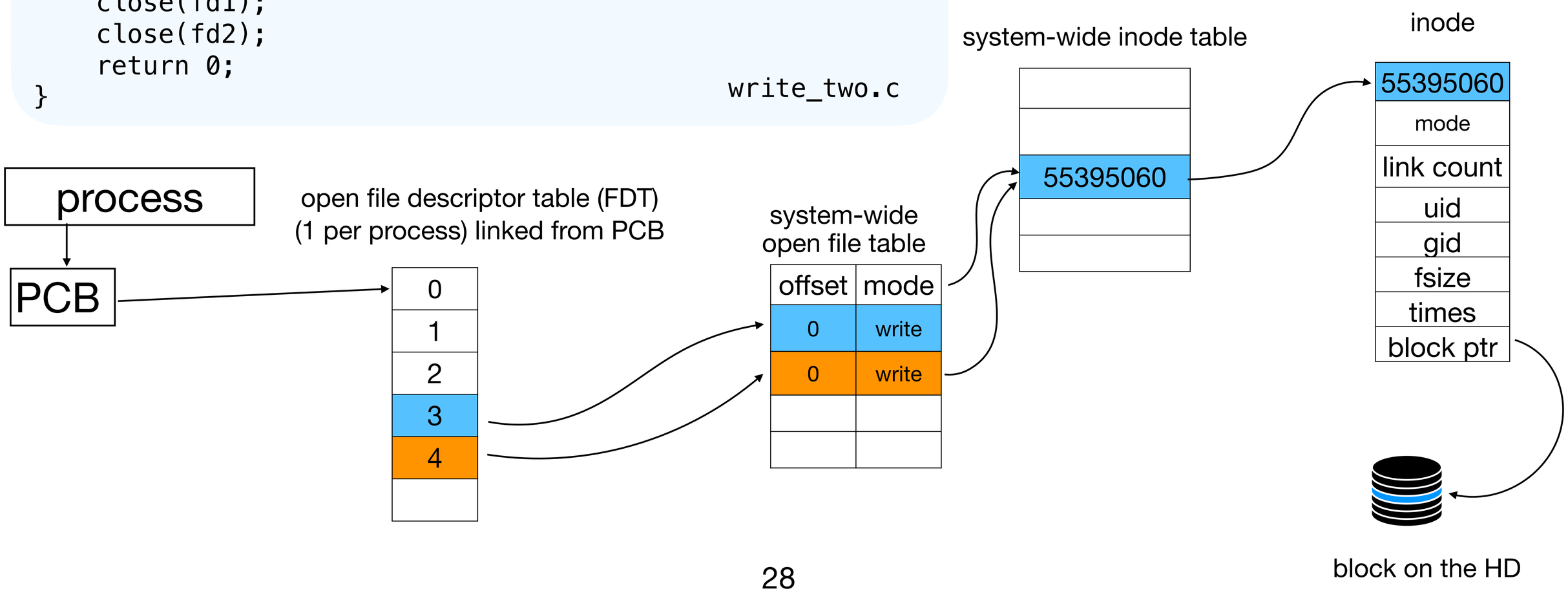
write BBBB into the second fd

write_two.c

```
>>> ./write_two
fd1 = 3
fd2 = 4
```

file.txt

BBBB



duplicate file descriptors

```
int main(void) {
    int fd1 = open("file.txt", O_CREAT | O_WRONLY, 0644);
    int fd2 = dup(fd1);
    printf("fd1 = %d\n", fd1);
    printf("fd2 = %d\n", fd2);
    write(fd1, "AAAA", 4);
    write(fd2, "BBBB", 4);
    close(fd1);
    close(fd2);
    return 0;
}
```

duplicate the first fd

print ids

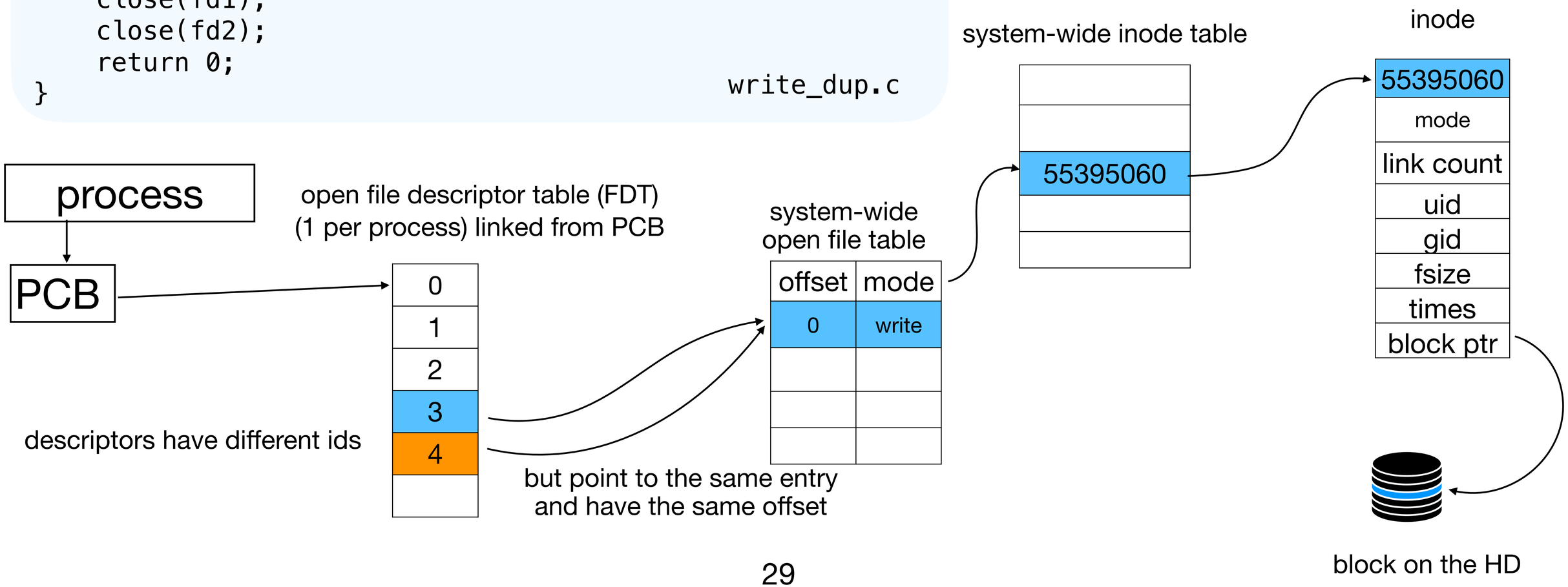
write two strings

write_dup.c

```
>>> ./write_dup
fd1 = 3
fd2 = 4
```

file.txt

AAAABBBB



fork with open file descriptor

```
int main(void) {
    int fd = open("file.txt", O_CREAT | O_WRONLY, 0644);
    if (fork() == 0) {
        sleep(1);
        printf("[child] fd = %d\n", fd);
        write(fd, "BBBB", 4);
    } else {
        printf("[parent] fd = %d\n", fd);
        write(fd, "AAAA", 4);
    }
    close(fd);
    return 0;
}
```

open a file descriptor

child sleeps and then prints

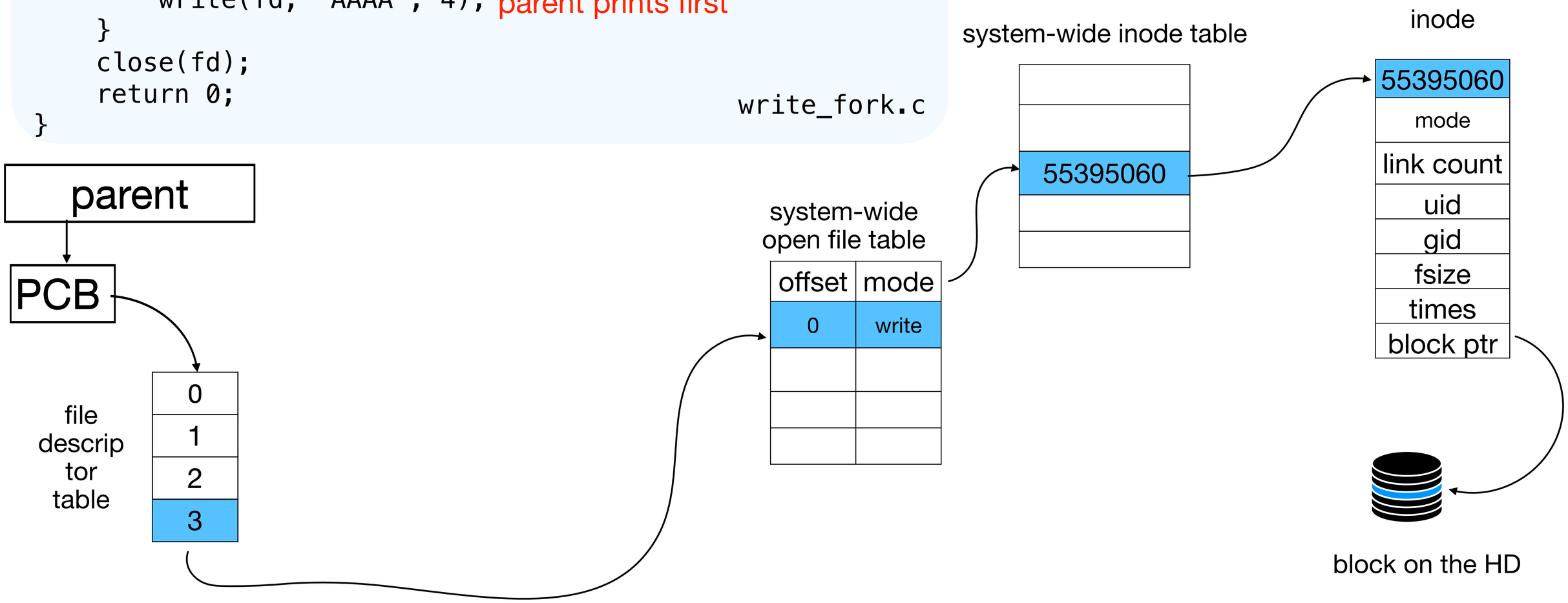
parent prints first

write_fork.c

```
>>> ./write_fork
[parent] fd = 3
[child] fd = 3
```

file.txt

AAAABBBB



fork with open file descriptor

```
int main(void) {
    int fd = open("file.txt", O_CREAT | O_WRONLY, 0644);
    if (fork() == 0) {
        sleep(1);
        printf("[child] fd = %d\n", fd);
        write(fd, "BBBB", 4);
    } else {
        printf("[parent] fd = %d\n", fd);
        write(fd, "AAAA", 4);
    }
    close(fd);
    return 0;
}
```

open a file descriptor

child sleeps and then prints

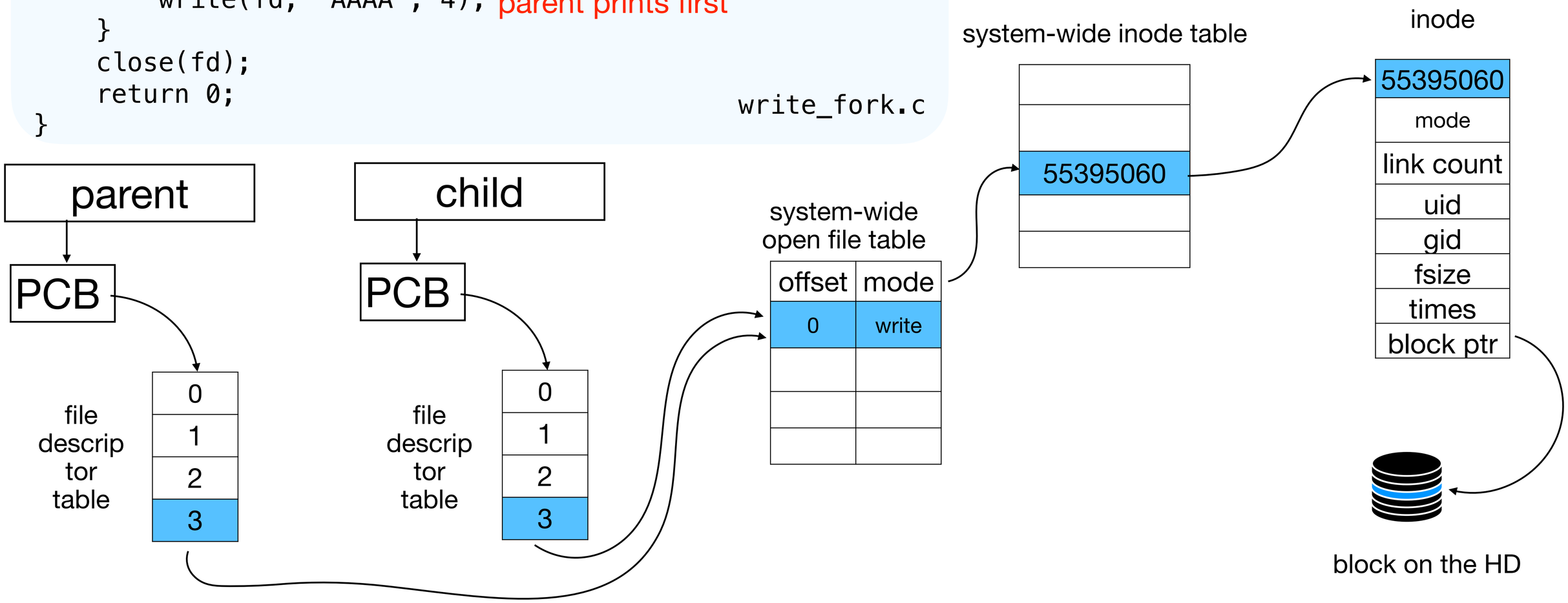
parent prints first

write_fork.c

```
>>> ./write_fork
[parent] fd = 3
[child] fd = 3
```

file.txt

AAAABBBB



Summary so far

- Each process has a table of open file descriptors
  - each new descriptor is added to the table
  - mapping is added to the system-wide open descriptor table
- File descriptors created with open() are **independent** of each other
  - have **different** offset and mode
  - point to the same inode
- File descriptors created with dup() are a **copy** of another descriptor
  - **share** offset and mode
  - point to the same inode

inode

`stat` command on Linux/Mac outputs file metadata
(stored in inode)

```
stat file.txt
  File: "file.txt"
  Size: 8
  FileType: Regular File
  Mode: (0644/-rw-r--r--)
  Uid: (  501/   yulia)
  Gid: (   20/   staff)
Device: 1,16
Inode: 55395060
Links: 1
Access: Thu Mar 19 16:38:42 2026
Modify: Thu Mar 19 16:38:42 2026
Change: Thu Mar 19 16:38:42 2026
Birth: Thu Mar 19 16:38:42 2026
```

size in bytes

permissions

id of the hard drive

group and user id of the owner

how many inodes point to this file

inode id

opened but not changed

file contents changed

file contents and/or metadata changed

file created

Directory inode

```
folder
├── file0.mp4
├── file1.txt
├── file2.bla
├── subfolder1
│   └── file3.txt
├── subfolder2
└── subfolder3
    ├── file5.txt
    └── file6.txt
```

can use `stat` command for directories too

```
stat file.txt
  File: "file.txt"
  Size: 8
  FileType: Regular File
  Mode: (0644/-rw-r--r--)
  Uid: (  501/   yulia)
  Gid: (   20/   staff)
Device: 1,16
Inode: 55395060
Links: 1
Access: Thu Mar 19 16:38:42 2026
Modify: Thu Mar 19 16:38:42 2026
Change: Thu Mar 19 16:38:42 2026
Birth: Thu Mar 19 16:38:42 2026
```

file type will be different

links are number of inodes pointing to the folder inode

```
stat folder
  File: "folder"
  Size: 224
  FileType: Directory
  Mode: (0755/drwxr-xr-x)
  Uid: (  501/   yulia)
  Gid: (   20/   staff)
Device: 1,16
Inode: 55391937
Links: 8
Access: Thu Mar 19 16:38:52 2026
Modify: Thu Mar 19 16:38:46 2026
Change: Thu Mar 19 16:38:46 2026
Birth: Thu Mar 19 16:11:52 2026
```

why are there 8?

Directory inode

```
folder
├── file0.mp4
├── file1.txt
├── file2.bla
├── subfolder1
│   └── file3.txt
├── subfolder2
└── subfolder3
    ├── file5.txt
    └── file6.txt
```

can use `ls -l` command to get a complete information about the file/folder

hard links

```
-rw-r--r--  1 yulia  staff    0 Mar 23 08:59 file0.mp4
-rw-r--r--  1 yulia  staff    0 Mar 19 16:38 file1.txt
-rw-r--r--  1 yulia  staff    0 Mar 19 16:38 file2.bla
drwxr-xr-x  2 yulia  staff   96 Mar 19 16:38 subfolder1
drwxr-xr-x  2 yulia  staff   96 Mar 19 16:39 subfolder2
drwxr-xr-x  2 yulia  staff  128 Mar 23 09:01 subfolder3
```

permissions user group size ctime - change time

subfolder1, 2, and 3 have different number of files but the same number of hard links

only subdirectories count as links

each folder has hard links for itself (.) and its parent (..)

Directory inode

folder has . and ..

3 links to each subfolder

2 + 3 + 3 = 8

each subfolder has a link to the parent (..)

```
folder
├── file0.mp4
├── file1.txt
├── file2.bla
├── subfolder1
│   └── file3.txt
├── subfolder2
└── subfolder3
    ├── file5.txt
    └── file6.txt
```

can use `stat` command for directories too

```
stat file.txt
  File: "file.txt"
  Size: 8
  FileType: Regular File
  Mode: (0644/-rw-r--r--)
  Uid: (  501/   yulia)
  Gid: (   20/   staff)
Device: 1,16
Inode: 55395060
Links: 1
Access: Thu Mar 19 16:38:42 2026
Modify: Thu Mar 19 16:38:42 2026
Change: Thu Mar 19 16:38:42 2026
Birth: Thu Mar 19 16:38:42 2026
```

```
stat folder
  File: "folder"
  Size: 224
  FileType: Directory
  Mode: (0755/drwxr-xr-x)
  Uid: (  501/   yulia)
  Gid: (   20/   staff)
Device: 1,16
Inode: 55391937
Links: 8
Access: Thu Mar 19 16:38:52 2026
Modify: Thu Mar 19 16:38:46 2026
Change: Thu Mar 19 16:38:46 2026
Birth: Thu Mar 19 16:11:52 2026
```

why are there 8?

Directory inode

- Directories **give names to inode numbers**
  - file metadata is stored in the file inode
- Directory inode stores directory metadata
  - the contents of the directory file is the **mapping** between filenames and inodes

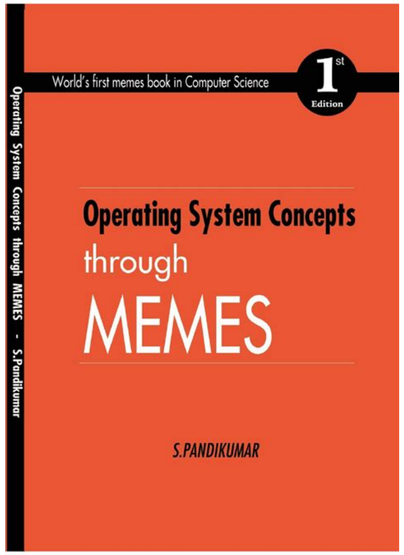
Directory inode

given an absolute path to the file

/cis3110/textbook.pdf

split the address into parts

/ cis3110 textbook.png



locate the root inode first

inode for root directory

2
mode
link count
uid
gid
fsize
times
block ptr

look for cis3110 in its mapping

contents of the root directory inode

.	2
..	2
cis3110	55663734
…	…

follow the pointer to the inode (if the directory is listed)

inode 55663734

55663734
mode
link count
uid
gid
fsize
times
block ptr

find the file inode (if it is listed)

contents of the cis3110 directory inode

.	55663734
..	1
textbook.png	55607504
…	…

follow the pointer to the inode and get the data

inode 55607504

55607504
mode
link count
uid
gid
fsize
times
block ptr

block ptr to the actual image bytes on hd

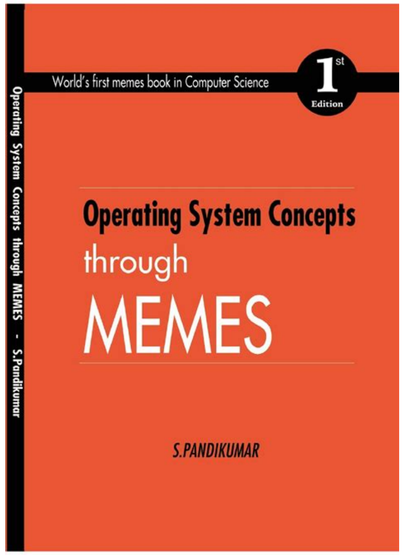
Directory inode

what if it is a relative path?

./cis3110/textbook.pdf

same process, but start with the current dir

names and contents for files and directories are stored separately!



OS will locate the cwd inode

inode for current directory

39265898
mode
link count
uid
gid
fsize
times
block ptr

look for cis3110 in its mapping

contents of the current directory inode

.	2
..	2
cis3110	55663734
…	…

follow the pointer to the inode (if the directory is listed)

inode 55663734

55663734
mode
link count
uid
gid
fsize
times
block ptr

find the file inode (if it is listed)

contents of the cis3110 directory inode

.	55663734
..	1
textbook.png	55607504
…	…

follow the pointer to the inode and get the data

inode 55607504

55607504
mode
link count
uid
gid
fsize
times
block ptr

block ptr to the actual image bytes on hd

Summary so far

- File path tells the OS how to navigate the system-wide inode table
  - path is split into parts (by a / delimiter)
  - OS finds the inode for the first part
  - if it is a directory
    - its contents will have inodes for all files and directories inside
  - if it is a file
    - its contents is the actual contents of the file
- If the path is incorrect, the OS will not be able to locate it

Names and data

- Names of files/directories are stored **separately** from their data
  - name of a file is stored in the inode of its directory
  - metadata of a file is stored in another inode
- Possible to have **different names** pointing to the **same inode**
  - hard and soft links

Links

- Make a file appear in **two locations at the same time**
- **Hard links** - a reference to the same inode, i.e. another path for the same file
  - C call - `int link(const char *target, const char *linkpath)`
  - in terminal - `ln target linkpath`
- **Soft (symbolic) links** - a reference to another path, like a **shortcut**
  - C call - `int symlink(const char *target, const char *linkpath)`
  - in terminal - `ln -s target linkpath`

Hard link

- Hard link is another **name** for the same **inode**
- Both links are the same
  - i.e. there is no "original"
- Files are created with 1 hard link
  - Can add any number of additional hard links to any file
- Directories start with 2 hard links (. and ..)
  - Cannot add more hard links

Hard link

```
>>> pwd
/home/yulia/CIS3110/folder
>>> ls -li file0.mp4
55968422 -rw-r--r--  1 yulia  staff  0 Mar 23 10:46 file0.mp4
>>> ln file0.mp4 subfolder2/file0.mp4
>>> ls -li subfolder2/file0.mp4
55968422 -rw-r--r--  2 yulia  staff  0 Mar 23 10:46 subfolder2/file0.mp4
```

hard link count increased

inodes of both files are the same

```
folder
├── file0.mp4
├── file1.txt
├── file2.bla
├── subfolder1
│   └── file3.txt
├── subfolder2
│   └── file0.mp4
└── subfolder3
    ├── file5.txt
    └── file6.txt
```

now this file has 2 paths!

this is the "original" file

folder inode 55962127

.	55962127
..	55391892
file0.mp4	55968422
file1.txt	55968424
file2.bla	55968431
subfolder1	55968435
subfolder2	55968444
subfolder3	55968625

subfolder2 inode

.	55968444
..	55962127
file0.mp4	55968422

folder/file0.mp4

folder/subfolder2/file0.mp4

file0.mp4 inode

55968422
mode
link count
uid
gid
fsize
times
block ptr

In creates a new entry in the subfolder2 inode pointing to file0mp4

Soft link

- aka symbolic link
- Has its own inode
  - what kind of inode and what is in it is OS-specific
  - Linux: symlink metadata and **path** to the original file

Soft link (continued from hard link example)

```
folder
├── file0.mp4
├── file1.txt
├── file2.bla
├── subfolder1
│   ├── file3.txt
│   └── file0.mp4
├── subfolder2
│   └── file0.mp4
└── subfolder3
    ├── file5.txt
    └── file6.txt
```

```
>>> ln -s file0.mp4 subfolder1/file0.mp4
ls -lai subfolder1
total 0
55968435 drwxr-xr-x  4 yulia  staff  128 Mar 23 13:14 .
55962127 drwxr-xr-x  8 yulia  staff  256 Mar 23 10:47 ..
56098898 lrwxr-xr-x  1 yulia  staff    9 Mar 23 13:14 file0.mp4 -> file0.mp4
55968438 -rw-r--r--  1 yulia  staff    0 Mar 23 10:47 file3.txt
```

you can tell it's a symbolic link from ls output

permissions string starts with "l"

this is where the link points to

```
  File: "subfolder1/file0.mp4"-> file0.mp4
  Size: 9
  FileType: Symbolic Link
  Mode: (0755/lrwxr-xr-x)
  Uid: (  501/   yulia)
  Gid: (   20/   staff)
Device: 1,16
Inode: 56098898
Links: 1
Access: Mon Mar 23 13:14:22 2026
Modify: Mon Mar 23 13:14:22 2026
Change: Mon Mar 23 13:14:32 2026
 Birth: Mon Mar 23 13:14:22 2026
```

stat will also show a different file type

Soft link (continued from hard link example)

```
>>> ln -s file0.mp4 subfolder1/file0.mp4
ls -lai subfolder1
total 0
55968435 drwxr-xr-x  4 yulia  staff  128 Mar 23 13:14 .
55962127 drwxr-xr-x  8 yulia  staff  256 Mar 23 10:47 ..
56098898 lrwxr-xr-x  1 yulia  staff    9 Mar 23 13:14 file0.mp4 -> file0.mp4
55968438 -rw-r--r--  1 yulia  staff    0 Mar 23 10:47 file3.txt
```

```
folder
├── file0.mp4
├── file1.txt
├── file2.bla
├── subfolder1
│   ├── file3.txt
│   └── file0.mp4
├── subfolder2
│   └── file0.mp4
└── subfolder3
    ├── file5.txt
    └── file6.txt
```

symlink has its own inode

folder inode 55962127

.	55962127
..	55391892
file0.mp4	55968422
file1.txt	55968424
file2.bla	55968431
subfolder1	55968435
subfolder2	55968444
subfolder3	55968625

subfolder1 inode

.	55968444
..	55962127
file0.mp4	56098898

file0.mp4 symlink inode

56098898
path to file0.mp4

symlink inode contains the path it was given at creation

folder/file0.mp4

subfolder2 inode

.	55968444
..	55962127
file0.mp4	55968422

folder/subfolder2/file0.mp4

55968422
mode
link count
uid
gid
fsize
times
block ptr

Hard vs soft links

every directory entry is a hard link

hard links point to an inode which is independent of the path

internals of inodes are FS-specific

can create cycles and break FS

permissions are stored in the inode and it is the same for all hard links

Hard links	Symbolic links
A record that links to file **inode**	A record that contains another file's **path**
Will **not break** when the original is deleted or moved	Will **break** when the original is moved or deleted
Can point only to the files in t**he same filesystem**	Can point to **any file**
Can be made **only to files**	Can be made **to files and directories**
Permissions are the **same** as those on the "original"	Symlinks can have **different** permissions from the original (on some OSs)

does not increment directory link count

soft links point to a path, which can change

links to path, not inode

can also create cycles but won't break the system

mkdir

- `int mkdir(const char *path, mode_t mode)`
  - C system call for creating directories
  - on success - returns 0
  - on failure - returns -1, sets `errno`
- `mkdir`
  - create directory via command line
- creates an inode for it
  - mapping is initialized with **two entries**
    - dot (.) - hard link to the current directory inode
    - dot dot (..) - hard link to the parent directory inode
- increases hard link count of its parent directory by 2
  - its own link and it’s link to the parent

unlink

- `int unlink(const char *path)`
- `path`
  - deletes a file from the filesystem
  - if any processes opened the file, it will remain until they exit
- on success
  - file’s inode is removed from the file system, space can be reused
  - if it is a symbolic link, it’s inode is removed
- on failure
  - returns -1 and sets errno

unlink

- `int unlink(const char *path)`
- why the name is not delete or remove?
  - when the file is created 2 things happen
    - an inode is created
    - inode is **linked** to its human-readable path and to the directory
  - removing a file **unlinks** it from the directory
    - hard link count is decremented for both directory and file
    - when file link count = 0, file is removed
- on success - returns 0
- on failure - returns -1, sets `errno`

everything is eventually linked to the root directory /

rm

- Delete files and directories via command line
  - **irreversible** and dangerous
  - deleted files **will not** go to trash bin
- Deleting files
  - `rm file.txt`
- Deleting folders
  - `rm folder` - will fail
  - `rm -r folder` - will delete the folder and its contents
  - `rm -rf folder` - will force delete the folder and its contents
  - **`rm -rf *` - will delete everything in the current directory**

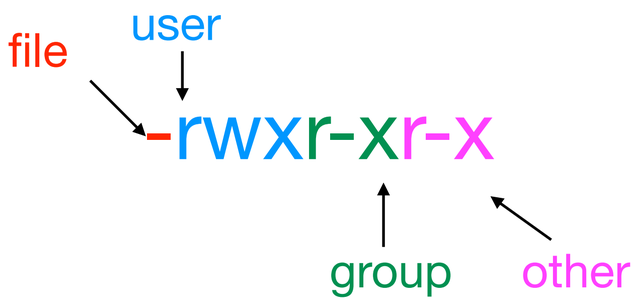
USE ONLY IF YOU KNOW WHAT YOU ARE DOING

DO NOT DO THIS FROM THE ROOT DIRECTORY

Permissions

r - read
w - write
x - execute
- - no permission

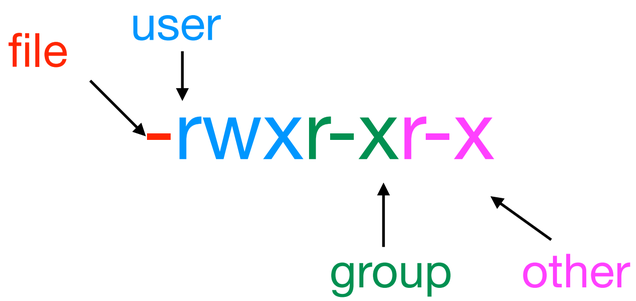
- aka mode
- represents
  - **file type** - file or directory
  - **permissions** for
    - user/group/other
    - to read/write/execute



Permissions

r - read
w - write
x - execute
- - no permission

- Can view permissions
  - `ls -l`
- Can change permissions
  - `chmod`



chmod

u - user
g - group
o - other
a - all (u+g+o)

r - read
w - write
x - execute
- - no permission

- Can change permissions
  - one by one
    - `chmod u=rwx file.txt` users can read/write/execute
    - `chmod g=rx file.txt` group can read/execute
    - `chmod o=rx file.txt` others can read/execute
  - all together
    - `chmod r=rwx,g=rx,o=rx file.txt` same thing in one line

file user

-rwxr-xr-x

group other

chmod

u - user
g - group
o - other
a - all (u+g+o)

r - read
w - write
x - execute
- - no permission

- Can add or remove permission bits
  - `chmod u+r file.txt` add read permissions to the user
  - `chmod g-w file.txt` remove write permissions from the group
  - `chmod a+x file.txt` give execute permissions to all

file

-rwxr-xr-x

chmod

u - user
g - group
o - other
a - all (u+g+o)

r - read
w - write
x - execute
- - no permission

- Alternative form
  - `chmod 755 file.txt` can do the same with numbers
  - `chmod 644 file.txt` more compact
- Can set permissions at file creation
  - `open("file.txt", O_WRONLY, 0755);`

file

symbolic -rwxr-xr-x

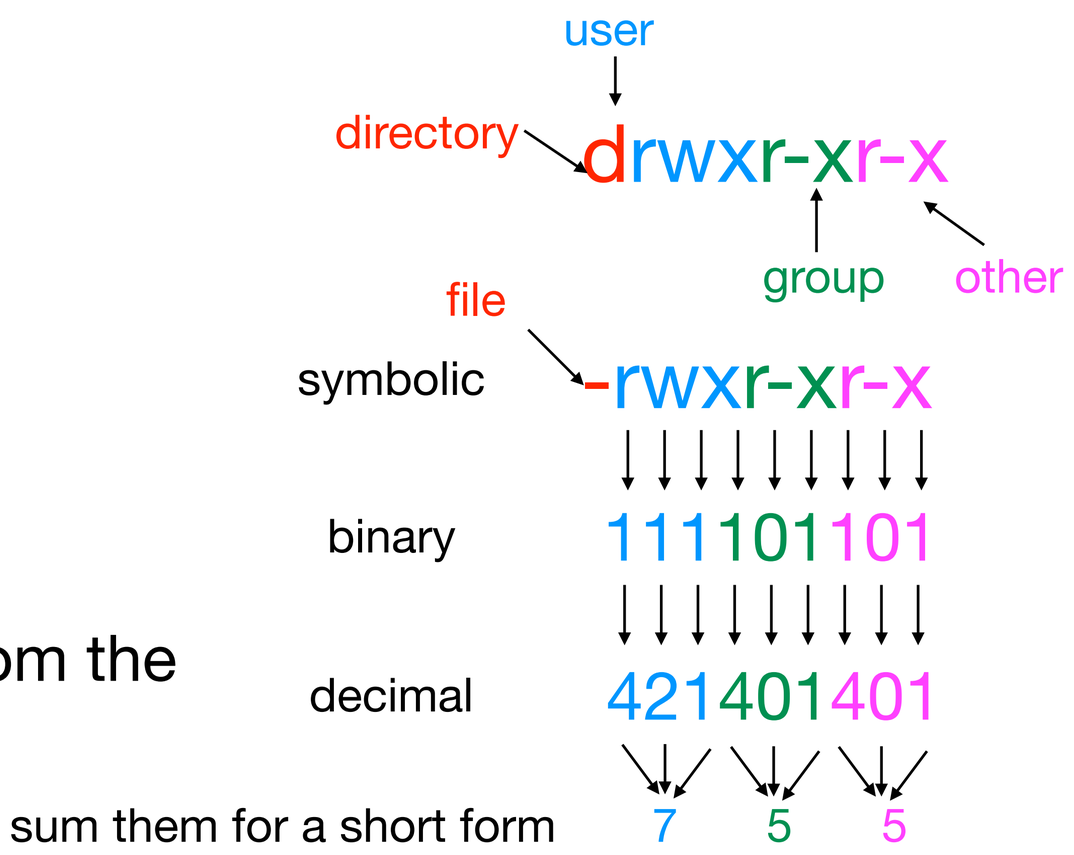
binary 111101101

decimal 421401401

sum them for a short form 7 5 5

Directories

- permissions
  - first character is set to d
- can use chmod to set/modify
  - same syntax as for files
- directory can have different permissions from the files it contains



Summary

- `open` - open/create a file
- `read/write` - copy bytes from/to memory
- `lseek` - get/set current location in the file
- `close` - stop using a file
- `unlink` - delete a file
- `link, symlink` - link to a file
- `rename` - change file name
- `mkdir/rmdir` - create/delete directories

Summary

- Each process tracks its open descriptors
- OS tracks all open file descriptors, their offsets and permissions
- File descriptors created with open() are **independent** of each other
  - have **different** offset and mode
  - point to the same inode
- File descriptors created with dup() are a **copy** of another descriptor
  - **share** offset and mode
  - point to the same inode

Summary

- Files - sequences of bytes stored on HD
  - name - human-readable identifier
  - inode - stores file metadata and links to HD
- Directories - similar to files
  - name - human-readable identifier
  - inode - stores mapping between files in the directory and their inodes

Summary

- Names and contents of the files/directories are stored **separately**
- **Hard links** - different paths for the same inode
- **Soft (symbolic) links** - different aliases for the same path

Reading

- Chapter 39 Files and directories
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf
- Chapter 40 HDD
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-implementation.pdf

Quiz 18

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999